Universidad Mayor de San Simón

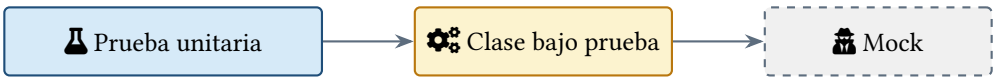
Facultad de Ciencias y Tecnología

Carrera de Ingeniería de Sistemas

Materia: Auditoría de Sistemas

Mockito

Dobles de prueba en Java: SRS, SDD y diseño paso a paso de pruebas de caja negra y caja blanca



Grupo 10

Main Rocabado Jairo Gael
Banda Tellez Fernando
Velasquez Ricaldez Rodrigo

Exposición: 29 de septiembre de 2026 08:15 – 08:44

Cochabamba – Bolivia 2026

Resumen

Mockito es el framework más usado en Java para crear *dobles de prueba* (mocks): objetos que reemplazan a las dependencias de una clase (base de datos, banco, correo) para probarla de forma aislada, rápida y repetible.

Este trabajo explica qué es Mockito, cómo está construido y cómo se usa. Lo aplica a un sistema de ejemplo, un *Servicio de Pedidos*, especificado con una **SRS** (IEEE 830 / ISO/IEC/IEEE 29148) y diseñado con un **SDD** (IEEE 1016). El aporte central es mostrar **paso a paso cómo diseñamos los casos de prueba**: de caja negra (clases de equivalencia, valores límite y tabla de decisión) y de caja blanca (grafo de flujo, complejidad ciclomática y camino básico). Se muestra también cómo se escribe una prueba desde que falla (rojo) hasta que pasa (verde).

Resultado: 65 pruebas automatizadas en verde y 100 % de cobertura de ramas en la lógica de negocio. Además, diseñar las pruebas reveló 4 vacíos en los requisitos, lo que muestra su valor como herramienta de **auditoría**.

Palabras clave: Mockito, pruebas unitarias, mocks, SRS, SDD, caja negra, caja blanca, diseño de casos de prueba.

Índice general

Resumen I

1. Introducción 1
- 1.1. Problema . . . 1
- 1.2. Objetivos . . . 1
- 1.3. Alcance . . . 1

2. Marco teórico 2
- 2.1. Pruebas unitarias y dobles de prueba . . . 2
- 2.2. Caja negra y caja blanca . . . 2
- 2.3. Ubicación en la evaluación de software . . . 2
- 2.4. Relación con la auditoría de sistemas . . . 3

3. Mockito 4
- 3.1. ¿Qué es? . . . 4
- 3.2. Arquitectura y componentes . . . 4
- 3.3. API principal . . . 5

4. SRS – Especificación de requisitos 6
- 4.1. Introducción y descripción general . . . 6
- 4.2. Interfaces externas . . . 6
- 4.3. Requisitos funcionales . . . 6
- 4.4. Reglas de negocio y caso de uso . . . 7
- 4.5. Requisitos no funcionales . . . 7
- 4.6. Observaciones de la revisión . . . 7

5. SDD – Documento de diseño 8
- 5.1. Contexto . . . 8
- 5.2. Diseño lógico . . . 9
- 5.3. Algoritmo principal . . . 9
- 5.4. Estrategia de pruebas y trazabilidad . . . 10

6. Diseño de los casos de prueba, paso a paso 11
- 6.1. Procedimiento general . . . 11
- 6.2. Cómo se escribe una prueba: de rojo a verde . . . 11
- 6.3. Caja negra paso a paso . . . 12
  - 6.3.1. Clases de equivalencia y valores límite (`calcularDescuento`) . . . 12
  - 6.3.2. Tabla de decisión (`procesarPedido`) . . . 13

6.4. Caja blanca paso a paso (`procesarPedido`) 14

7. Resultados **17**

8. Conclusiones y recomendaciones **18**

Referencias **19**

A. Proyecto y ejecución **20**

Capítulo 1

Introducción

1.1 Problema

Una clase casi nunca trabaja sola. Nuestro `PedidoService` consulta el inventario, cobra mediante un banco y envía correos. Probarlo con esos sistemas reales hace que la prueba sea **lenta**, **frágil** (si el banco cae, falla), que tenga **efectos reales** (cobros y correos) y que no se puedan provocar a voluntad casos como "pago rechazado". Los **dobles de prueba** resuelven esto, y Mockito es la herramienta estándar para crearlos.

1.2 Objetivos

General: explicar el funcionamiento y el uso de Mockito y aplicarlo, paso a paso, al diseño de pruebas de caja negra y de caja blanca de un sistema especificado con SRS y SDD.

Específicos:

1. Describir qué es Mockito, su arquitectura y sus componentes principales.
2. Elaborar la SRS y el SDD de un sistema de ejemplo diseñado para ser probado.
3. Mostrar paso a paso cómo se diseñan y se escriben los casos de prueba de caja negra y de caja blanca.
4. Ejecutar las pruebas, medir la cobertura y relacionar los resultados con la auditoría de sistemas.

1.3 Alcance

Pruebas **unitarias** en Java 21 con JUnit 6 y Mockito 5.24. El código está en la carpeta `ejemplos-mockito/` y se ejecuta con `./run.sh` (anexo A).

Capítulo 2

Marco teórico

2.1 Pruebas unitarias y dobles de prueba

Una **prueba unitaria** verifica una clase o un método aislado de sus dependencias. Para aislarla se usan **dobles de prueba** (Meszaros, 2007):

Tabla 2.1: Tipos de dobles de prueba

Tipo	Qué hace	En Mockito
Stub	Devuelve respuestas predefinidas	`when(...).thenReturn(...)`
Mock	Registra las llamadas para verificarlas después	`verify(...)`
Spy	Objeto real que además registra su uso	`spy(obj)`
Dummy / Fake	Relleno de parámetros / implementación simplificada	se crean a mano

Un criterio importante: **no se simula la lógica propia que se quiere comprobar**. Se simulan las dependencias externas o colaboradoras (inventario, pagos, notificaciones y persistencia), tal como se define en el SDD.

2.2 Caja negra y caja blanca

Tabla 2.2: Enfoques de diseño de pruebas

	Caja negra	Caja blanca
Se diseña desde	los requisitos (SRS), sin ver el código	el código fuente
Técnicas	clases de equivalencia, valores límite, tabla de decisión	grafo de flujo, complejidad ciclomática, camino básico
Pregunta	¿hace lo que se pidió?	¿se probó todo lo que se escribió?

La **complejidad ciclomática** de McCabe indica el número de caminos independientes de un método, es decir, el número mínimo de casos de prueba de caja blanca:

$$V(G) = E - N + 2 \qquad V(G) = P + 1 \qquad V(G) = R$$

donde E son aristas, N nodos, P nodos de decisión y R regiones del grafo.

2.3 Ubicación en la evaluación de software

Según la clasificación vista en la materia:

- **Evaluación estática** (revisiones e inspecciones, sin ejecutar): la aplicamos a la SRS y al SDD con una lista de comprobación (sección 4.6).
- **Evaluación dinámica** (pruebas, ejecutando el código): Mockito se usa en el nivel de **prueba de unidad**, antes de la integración, el sistema y la aceptación.

- **Verificación** (¿se construyó bien?) con las pruebas unitarias; **validación** (¿es lo que el cliente necesita?) con los criterios de aceptación de la SRS.
- En ISO/IEC 25010, la *capacidad para ser probado* (mantenibilidad) es la que el SDD diseña y Mockito aprovecha.

La cadena **error** → **defecto** → **fallo** explica por qué revisamos la SRS: si el analista se equivoca (error) y escribe mal el descuento VIP en la SRS (defecto), el cliente pagará de más (fallo). Una prueba de caja negra, cuyo resultado esperado sale de esa misma SRS, *no* lo detectaría: las pruebas demuestran la presencia de defectos, no su ausencia.

2.4 Relación con la auditoría de sistemas

Para el auditor, las pruebas automatizadas son **evidencia**: son repetibles, se relacionan con los requisitos (matriz de trazabilidad) y su alcance se mide con la cobertura. Marcos como COBIT 2019 (BAI03) e ISO/IEC/IEEE 29119 exigen pruebas planificadas y documentadas.

Capítulo 3

Mockito

3.1 ¿Qué es?

Mockito es un framework de código abierto (licencia MIT) para crear mocks, stubs y spies en Java. Lo creó Szczepan Faber en 2008 y hoy es el estándar de facto: viene incluido en Spring Boot. Usamos la versión **5.24.0**, que requiere Java 11 o superior. Se instala agregando `mockito-core` y `mockito-junit-jupiter` al `pom.xml`.

3.2 Arquitectura y componentes

Mockito se organiza en capas: el tester solo usa la API pública; por debajo, un núcleo interno fabrica los mocks e intercepta sus llamadas, apoyado en las bibliotecas Byte Buddy y Objenesis.

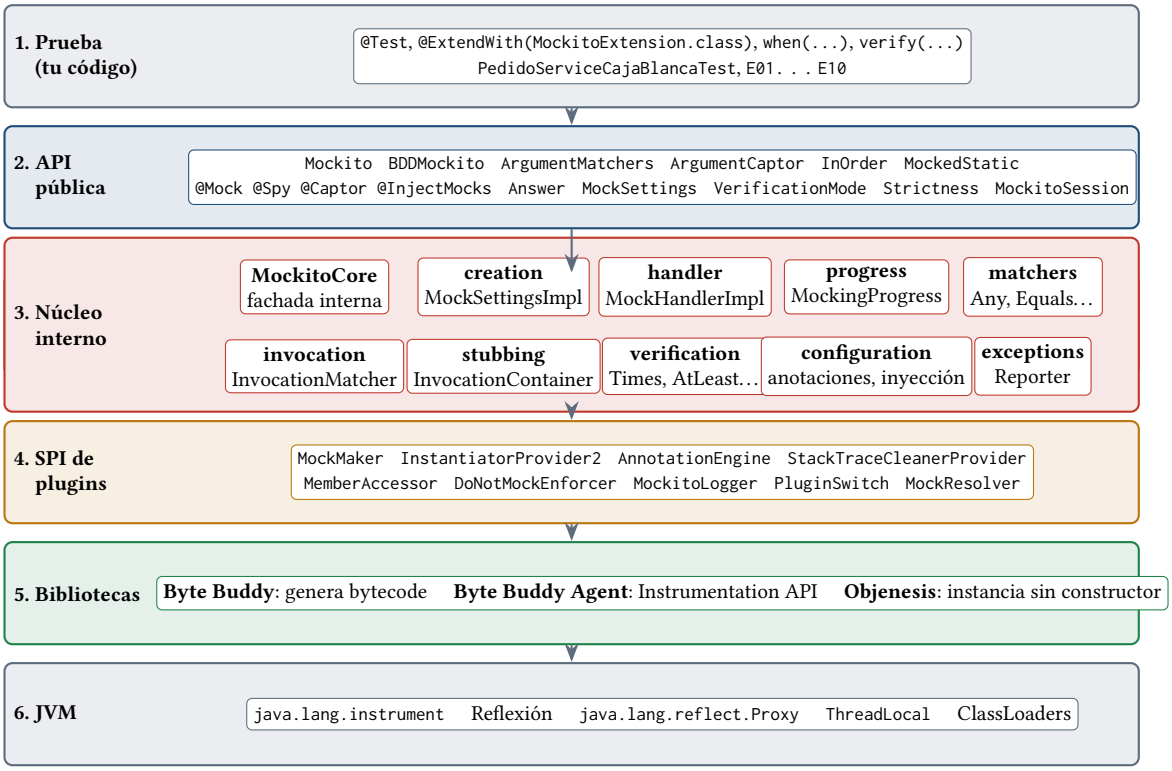


Figura 3.1: Arquitectura de Mockito por capas

Tabla 3.1: Componentes principales

Componente	Responsabilidad
`Mockito` (fachada)	Punto de entrada: `mock`, `when`, `verify`… Delega en `MockitoCore`.
`MockMaker` + Byte Buddy	Fabrica el mock. Por defecto (*inline*) reescribe el bytecode de la clase, lo que permite simular clases `final` y métodos estáticos.
Objenesis	Crea el objeto mock **sin ejecutar su constructor**.
`MockHandler`	Recibe **cada llamada** al mock: la registra y decide la respuesta.
`InvocationContainer`	Guarda las llamadas recibidas (para `verify`) y los stubs programados.
`MockingProgress`	Memoria por hilo: recuerda la **última llamada** a un mock y los *matchers* pendientes.
`Answer`	Decide qué devuelve el mock. Sin stub: `0`, `false`, `null` o colecciones vacías.
`VerificationMode`	Implementa `times(n)`, `never()`, `atLeast(n)`…

Cómo funciona `when(inv.stockDisponible("P1")).thenReturn(5)`.

1. Java ejecuta primero `inv.stockDisponible("P1")`; la llamada llega al `MockHandler`.
2. El handler la registra y avisa a `MockingProgress`: "esta es la última llamada". Como no hay stub, devuelve `0`.
3. `when(0)` ignora el `0` y toma de `MockingProgress` esa última llamada.
4. `thenReturn(5)` la guarda como stub. Desde entonces, esa llamada devuelve `5`.

De este mecanismo salen las reglas prácticas de Mockito: si un argumento usa un *matcher* (`any()`, `eq()`), **todos** deben usarlo; y los métodos `void` se programan con `doThrow()/doNothing().when(mock)`.

3.3 API principal

Tabla 3.2: Resumen de la API

API	Uso
`@Mock`, `@Spy`, `@InjectMocks`	Crear mocks/spies e inyectarlos en la clase bajo prueba
`when(...).thenReturn / thenThrow / thenAnswer`	Programar respuestas (stubs)
`doThrow / doNothing / doAnswer().when(mock)`	Programar métodos `void`
`verify(mock, times(n) | never())`	Verificar llamadas
`any()`, `eq()`, `anyString()`, `argThat()`	Comodines de argumentos (*matchers*)
`ArgumentCaptor`, `InOrder`, `mockStatic`	Capturar argumentos, verificar el orden, simular estáticos

El proyecto tiene 10 ejemplos didácticos de esta API (`E01` a `E10`, carpeta `basico/`).

Capítulo 4

SRS – Especificación de requisitos

Estructura según IEEE 830 / ISO/IEC/IEEE 29148. La SRS dice **qué** debe hacer el sistema y es la **base de las pruebas de caja negra**.

4.1 Introducción y descripción general

Propósito: especificar el *Servicio de Pedidos* de una tienda en línea. **Alcance:** recibe un pedido, verifica el stock, calcula el descuento, cobra, actualiza el inventario, registra el pedido y notifica al cliente. La interfaz de usuario, la base de datos, el banco y el correo son sistemas externos. **Restricciones:** Java 21; montos en Bs con 2 decimales; las pruebas no pueden usar sistemas reales.

4.2 Interfaces externas

Tabla 4.1: Interfaces externas (se reemplazarán por mocks)

ID	Interfaz	Operaciones
IE-01	Inventario	`stockDisponible(codigo)`, `descontarStock(codigo, cantidad)`
IE-02	Pasarela de pagos	`cobrar(email, monto)` → aprobado / rechazado
IE-03	Notificador	`enviar(destinatario, mensaje)`
IE-04	Registro de pedidos	`guardar(pedido)`, `buscarPorId(id)`

4.3 Requisitos funcionales

Tabla 4.2: Requisitos funcionales y criterios de aceptación

ID	Requisito	Criterio de aceptación (Dado / Cuando / Entonces)
RF-01	Rechazar pedidos sin productos.	Dado un pedido sin items, cuando se procesa, entonces se produce un error y no se cobra.
RF-02	Verificar el stock de cada item antes de cobrar (RN-03).	Dado un item con cantidad 2 y stock 1, entonces hay error de stock y no se cobra. Con stock = cantidad, el stock alcanza.
RF-03	Calcular el descuento (RN-01, RN-02, RN-04).	Dado un subtotal de 500 Bs, el descuento es 50,00 Bs (REGULAR) o 75,00 Bs (VIP).
RF-04	Si el pago es rechazado: estado RECHAZADO, notificar el motivo y no tocar el inventario.	Dado un banco que rechaza con “Sin fondos”, entonces el estado es RECHAZADO, se envía “Pago rechazado: Sin fondos” y no se descuenta stock.

ID	Requisito	Criterio de aceptación
RF-05	Si el pago es aprobado: descontar stock, estado PAGADO, registrar y confirmar.	Dado un pedido de 2 monitores de 250 Bs (REGULAR) y un banco que aprueba, entonces se cobran 450 Bs, el estado es PAGADO y se descuentan 2 unidades.
RF-06	Consultar un pedido por su identificador.	Dado un id inexistente, entonces se produce el error “No existe el pedido”.

4.4 Reglas de negocio y caso de uso

Tabla 4.3: Reglas de negocio

RN-01	Descuento por subtotal: menor que 100 Bs $\rightarrow$ 0 %; de 100 a 499,99 Bs $\rightarrow$ 5 %; desde 500 Bs $\rightarrow$ 10 %. Un subtotal negativo es inválido.
RN-02	Cliente VIP: 5 puntos porcentuales adicionales en cualquier tramo.
RN-03	El stock alcanza si *stock* $\geq$ *cantidad*.
RN-04	Total = subtotal $-$ descuento, redondeado a 2 decimales.

CU-01 Procesar pedido. *Flujo básico:* (1) se recibe el pedido; (2) se verifica que tenga items; (3) se verifica el stock de cada item; (4) se calcula el total; (5) se cobra; (6) el banco aprueba; (7) se descuenta el stock, se marca PAGADO, se registra y se notifica. *Alternativos:* FA-1 sin items $\rightarrow$ error; FA-2 sin stock $\rightarrow$ error, sin cobrar; FA-3 pago rechazado $\rightarrow$ RECHAZADO y notificación.

4.5 Requisitos no funcionales

RNF-01 **Testabilidad**: las dependencias externas se reciben como interfaces por constructor. RNF-02 **Calidad**: cobertura de ramas de la lógica de negocio $\geq$ 90 %. RNF-03 **Aislamiento**: las pruebas no acceden a sistemas reales.

4.6 Observaciones de la revisión

Revisamos la SRS con una **lista de comprobación** (evaluación estática): ¿es correcta?, ¿está completa?, ¿es consistente?, ¿es ambigua?, ¿cada requisito tiene diseño y prueba (trazabilidad)? Encontramos estos defectos:

Tabla 4.4: Hallazgos sobre los requisitos

OBS-01	No se especifica qué hacer con un pedido **nulo** (el código lo rechaza). Lo detectó la caja blanca.
OBS-02	No se validan cantidades 0 o negativas.
OBS-03	No se define el comportamiento con tipo de cliente nulo.
OBS-04	No se define qué hacer si el registro falla después de cobrar.

Capítulo 5

SDD – Documento de diseño

Estructura según IEEE 1016. El SDD dice **cómo** se construye el sistema para cumplir la SRS. La decisión principal es la **testabilidad**: todo sistema externo se accede por una interfaz inyectada, que en las pruebas se reemplaza por un mock.

5.1 Contexto

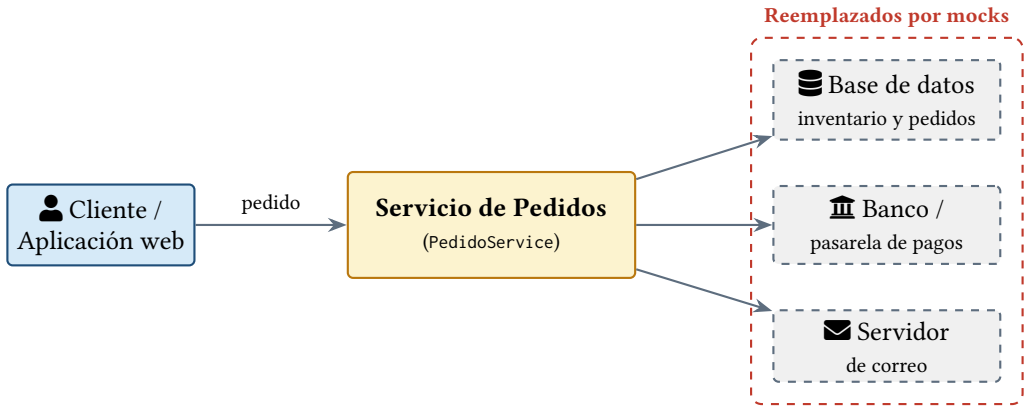


Figura 5.1: Diagrama de contexto. Los sistemas externos (línea discontinua) se sustituyen por mocks en las pruebas.

5.2 Diseño lógico

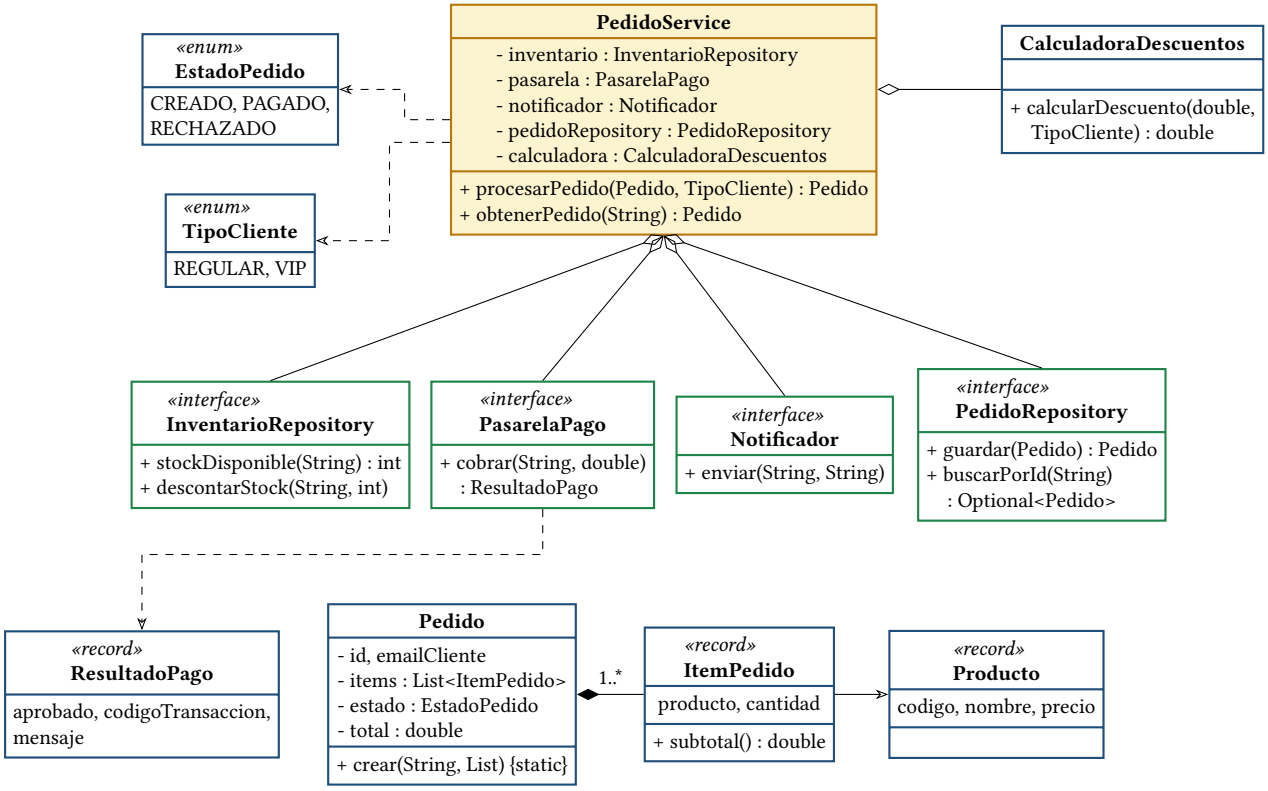


Figura 5.2: Diagrama de clases del Servicio de Pedidos (interfaces en verde = puertos *mockeables*)

5.3 Algoritmo principal

El método `procesarPedido` implementa CU-01. Sus decisiones (D1 a D6) son la base de las pruebas de caja blanca.

```
public Pedido procesarPedido(Pedido pedido, TipoCliente tipoCliente) {
    if (pedido == null || pedido.getItems().isEmpty()) {                // D1, D2
        throw new IllegalArgumentException("El pedido no tiene items");
    }
    double subtotal = 0;
    for (ItemPedido item : pedido.getItems()) {                         // D3
        String codigo = item.producto().codigo();
        if (inventario.stockDisponible(codigo) < item.cantidad()) {     // D4
            throw new StockInsuficienteException(codigo);
        }
        subtotal += item.subtotal();
    }
    double descuento = calculadora.calcularDescuento(subtotal, tipoCliente);
    double total = subtotal - descuento;

    ResultadoPago pago = pasarela.cobrar(pedido.getEmailCliente(), total);
    if (!pago.aprobado()) {                                             // D5
        pedido.setEstado(EstadoPedido.RECHAZADO);
        notificador.enviar(pedido.getEmailCliente(), "Pago rechazado: " + pago.mensaje());
        return pedido;
    }
    for (ItemPedido item : pedido.getItems()) {                         // D6
```

```
            inventario.descontarStock(item.producto().codigo(), item.cantidad());
        }
        pedido.setEstado(EstadoPedido.PAGADO);
        pedido.setTotal(total);
        Pedido guardado = pedidoRepository.guardar(pedido);
        notificador.enviar(pedido.getEmailCliente(), "Pedido " + pedido.getId()
                + " confirmado. Total: " + total + " Bs. Transacción: " + pago.codigoTransaccion());
        return guardado;
    }
```

Código 5.1: Método `procesarPedido` con sus decisiones marcadas (D1–D6)

5.4 Estrategia de pruebas y trazabilidad

Tabla 5.1: Qué se simula en las pruebas

Dependencia	Doble	Motivo
Inventario, Registro	Mock	Base de datos: fijamos el stock y el guardado
Pasarela de pagos	Mock	Evita cobros reales; permite simular un rechazo
Notificador	Mock	Evita correos reales; permite verificar el mensaje
`CalculadoraDescuentos`	Real / Spy / Mock	Lógica propia: real en caja negra; mock en caja blanca para aislar `procesarPedido`

Tabla 5.2: Matriz de trazabilidad: requisito → diseño → pruebas

Requisito	Diseño	Casos de prueba
RF-01	`procesarPedido` (D2)	R1 (negra); C2 (blanca)
OBS-01	`procesarPedido` (D1)	C1 (blanca)
RF-02	`procesarPedido` (D3, D4)	R2, L1 (negra); C3, C4 (blanca)
RF-03	`calcularDescuento`	CD-01 a CD-15 (negra); K1 a K5 (blanca)
RF-04	`procesarPedido` (D5)	R3 (negra); C5 (blanca)
RF-05	`procesarPedido` (D6)	R4 (negra); C6, C7 (blanca)
RF-06	`obtenerPedido`	E02
RNF-02	–	Reporte JaCoCo (capítulo 7)

Capítulo 6

Diseño de los casos de prueba, paso a paso

Este capítulo muestra **cómo** obtuvimos cada caso de prueba, no solo el resultado.

6.1 Procedimiento general

Seguimos el proceso de la norma ISO/IEC/IEEE 29119-4:

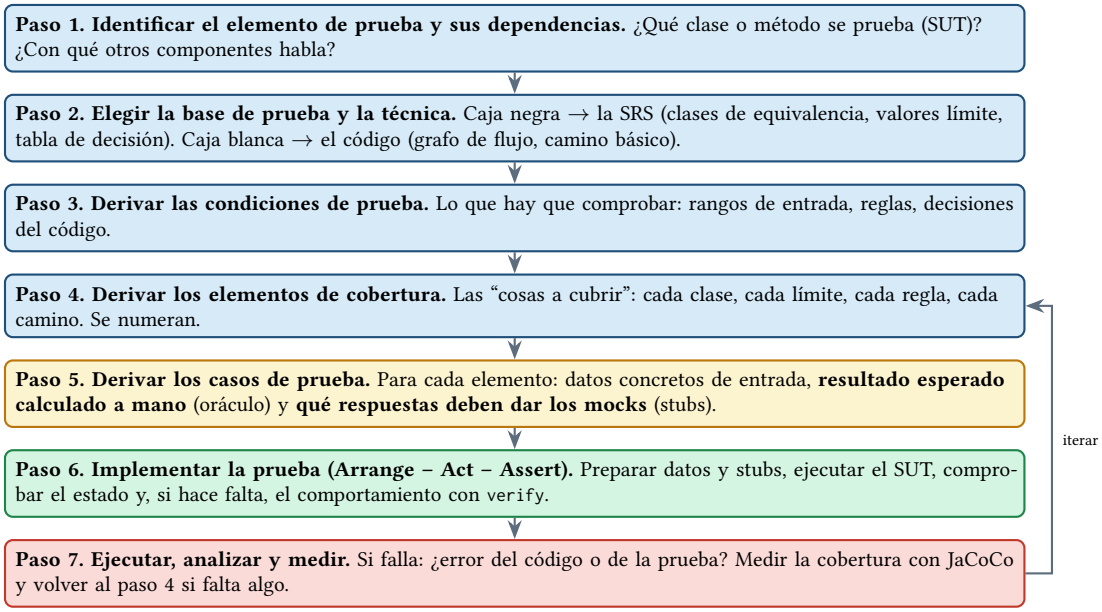


Figura 6.1: Proceso que seguimos para cada caso de prueba

Cada caso se documenta con una **ficha** (IEEE 829) *antes* de programarlo. La ficha registra el origen, los datos de entrada, qué responde cada mock, el resultado esperado calculado a mano y lo que se verifica.

6.2 Cómo se escribe una prueba: de rojo a verde

Tomamos el criterio de aceptación de RF-05 y construimos la prueba por versiones (paquete `pasoapaso`; las versiones en rojo se ejecutan con `./run.sh rojos`).

Tabla 6.1: Construcción de la prueba de RF-05, versión por versión

Versión	Qué agregamos	Resultado real	Qué aprendimos
V1	`@Mock` para cada dependencia externa y `@InjectMocks` para el servicio	verde	Mockito construyó el servicio con los mocks.
V2	Datos + llamada + `assertEquals(PAGADO,...)`, sin stubs	rojo: "Stock insuficiente para MON-01"	Un mock sin programar devuelve `0`: el stock valía 0.
V3	Stub del stock (`thenReturn(10)`)	rojo: `NullPointerException` en `pago.aprobado()`	`cobrar()` sin programar devolvió `null`. Hay que programar *todo* el camino.
V4	Stubs del cobro (450 Bs exactos) y del guardado	verde	El total esperado se calcula a mano *antes*: $500 - 10\ \% = 450$.
V5	`verify` de descuento de stock, guardado y correo	verde	Además del estado se verifica el comportamiento.
V6	Error del tester: esperar un cobro de 500	rojo: "Wanted 500.0 … but was 450.0"	La prueba *sí detecta* errores; el mensaje dice qué difiere.

```java
// ARRANGE: datos + un stub por cada consulta que hace el camino
Pedido pedido = new Pedido("PED-1", "cliente@umss.edu.bo", List.of(new ItemPedido(monitor, 2)));
when(inventario.stockDisponible("MON-01")).thenReturn(10);
when(pasarela.cobrar("cliente@umss.edu.bo", 450.0)).thenReturn(ResultadoPago.aprobado("TX-1"));
when(pedidoRepository.guardar(any(Pedido.class))).thenAnswer(inv -> inv.getArgument(0));
// ACT
Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.REGULAR);
// ASSERT: estado y comportamiento
assertEquals(EstadoPedido.PAGADO, resultado.getEstado());
assertEquals(450.0, resultado.getTotal(), 0.001);
verify(inventario).descontarStock("MON-01", 2);
verify(notificador).enviar(eq("cliente@umss.edu.bo"), startsWith("Pedido PED-1 confirmado"));
```

Código 6.1: Versión final (V5) con la estructura Arrange – Act – Assert

6.3 Caja negra paso a paso

6.3.1 Clases de equivalencia y valores límite (`calcularDescuento`)

Paso 1 – Elemento: `calcularDescuento(subtotal, tipo)`. No tiene dependencias, así que **no necesita mocks**.

Paso 2 – Base de prueba: RF-03 y las reglas RN-01, RN-02 y RN-04 de la SRS.

Paso 3 – Entradas y salidas: `subtotal` (Bs) y `tipo` (REGULAR/VIP); la salida es el descuento o un error.

Paso 4 – Clases y límites. Regla de Myers: un rango se divide en una clase por tramo más la zona inválida; un conjunto de valores da una clase por valor. En cada frontera b se prueban b y $b - 0{,}01$.

Tabla 6.2: Clases de equivalencia y valores límite

Clase	Condición	Tipo	Salida	Límites
CE1	subtotal < 0	inválida	error	$-0{,}01$
CE2	$0 \leq$ subtotal < 100	válida	0 %	0 y 99,99
CE3	$100 \leq$ subtotal < 500	válida	5 %	100 y 499,99
CE4	subtotal ≥ 500	válida	10 %	500
CE5 / CE6	REGULAR / VIP	válidas	+0 / +5 %	–

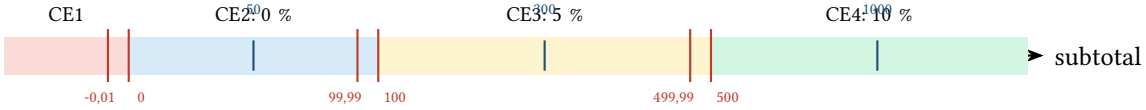


Figura 6.2: Particiones, valores límite (rojo) y representantes (azul)

Paso 5 – Casos y resultado esperado (calculado a mano). REGULAR se combina con todos los representantes y límites; VIP, con cada tramo al menos una vez (RN-02 es aditiva); cada caso inválido cubre una sola clase inválida.

Tabla 6.3: Casos de caja negra de `calcularDescuento`

ID	Clases	Subtotal	Tipo	Esperado (cálculo)
CD-01 a 03	CE2, CE5	50 / 0 / 99,99	REG	0,00
CD-04	CE3, CE5	100	REG	$100 \times 0{,}05 = 5{,}00$
CD-05	CE3, CE5	300	REG	15,00
CD-06	CE3, CE5	499,99	REG	$24{,}9995 \rightarrow 25{,}00$ (redondeo)
CD-07 / 08	CE4, CE5	500 / 1000	REG	50,00 / 100,00
CD-09 a 12	CE2–CE4, CE6	50 / 100 / 500 / 2000	VIP	2,50 / 10,00 / 75,00 / 300,00
CD-13 / 14	CE1	$-0{,}01$ / -100	REG	error
CD-15	CE5, CE6	varios	ambos	VIP siempre mayor que REGULAR

Paso 6 – Código. Como todos los casos válidos tienen la misma forma, cada fila de la tabla es una fila de un `@CsvSource`:

```
@ParameterizedTest(name = "{0}: subtotal={1}, {2} -> {3}")
@CsvSource({
    "CE2 límite superior,  99.99,  REGULAR, 0.00",    // CD-03
    "CE3 límite superior,  499.99, REGULAR, 25.00",   // CD-06
    "CE4 límite inferior,  500,    REGULAR, 50.00",   // CD-07
    "CE4 VIP límite inf.,  500,    VIP,     75.00"    // CD-11   ... (12 filas)
})
void clasesValidasYValoresLimite(String caso, double subtotal, TipoCliente tipo, double esperado)
     {
    assertEquals(esperado, calculadora.calcularDescuento(subtotal, tipo), 0.001, caso);
}
```

Código 6.2: Prueba parametrizada: una fila por caso

6.3.2 Tabla de decisión (`procesarPedido`)

Paso 1 – Condiciones y acciones, tomadas del CU-01: C1 ¿tiene items?, C2 ¿hay stock?, C3 ¿pago aprobado? Acciones: A1 error sin items, A2 error de stock, A3 RECHAZADO, A4 PAGADO.

Paso 2 – Tabla completa y simplificación. Con 3 condiciones hay $2^3 = 8$ reglas. Las que tienen la misma acción y difieren solo en condiciones irrelevantes se fusionan. Si no hay items, el stock y el pago no importan; si no hay stock, el pago no importa.

Tabla 6.4: De 8 reglas a 4

	1	2	3	4	5	6	7	8	**R1**	**R2**	**R3**	**R4**
C1 items	N	N	N	N	S	S	S	S	N	S	S	S
C2 stock	N	N	S	S	N	N	S	S	–	N	S	S
C3 pago	N	S	N	S	N	S	N	S	–	–	N	S
Acción	A1	A1	A1	A1	A2	A2	A3	A4	A1	A2	A3	A4

Paso 3 – Valor límite. C2 esconde la comparación de RN-03 (“$\geq$”), así que se agrega el caso **L1**: stock = cantidad, que debe aprobarse.

Paso 4 – Cómo se fuerza cada condición con Mockito. C1 se controla con los *datos* (pedido con o sin items). C2 y C3 dependen de sistemas externos, así que se fuerzan con *stubs*: `stockDisponible` $\rightarrow$ 1 o 10, y `cobrar` $\rightarrow$ rechazado o aprobado. En caja negra no se usa `verify`: solo se comprueban las salidas que promete la SRS.

Tabla 6.5: Casos de la tabla de decisión (pedido de 2 monitores de 250 Bs)

Caso	**Datos**	**Stubs**	**Esperado**
R1	pedido sin items	ninguno	error sin items
R2	2 unidades	stock $\rightarrow$ 1	error de stock
R3	2 unidades	stock $\rightarrow$ 10; cobrar $\rightarrow$ rechazado	RECHAZADO
R4	2 unidades	stock $\rightarrow$ 10; cobrar $\rightarrow$ aprobado; guardar	PAGADO, total 450
L1	2 unidades	stock $\rightarrow$ 2; cobrar $\rightarrow$ aprobado; guardar	PAGADO

Ficha R4 – Pago aprobado

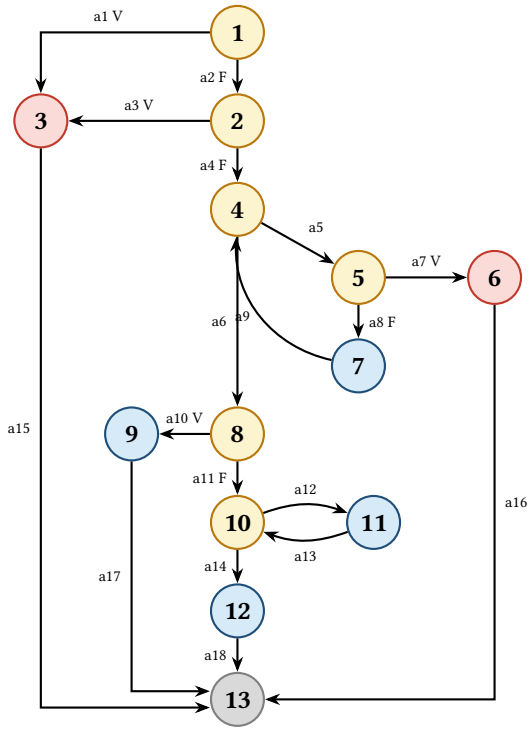
Origen / técnica	Caja negra, tabla de decisión, regla R4. RF-05.
Datos de entrada	`pedidoDe(2)`: 2 monitores de 250 Bs; cliente REGULAR.
Stubs (mocks)	`stockDisponible("MON-01")` $\rightarrow$ 10; `cobrar(...)` $\rightarrow$ `aprobado("TX-1")`; `guardar(...)` $\rightarrow$ el mismo pedido.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado PAGADO; total $= 500 - 10\ \% = 450{,}00$.
Verificaciones	`assertEquals(PAGADO, ...)`, `assertEquals(450.0, ...)`.
Resultado obtenido	Coincide con lo esperado: **APROBADO**

6.4 Caja blanca paso a paso (`procesarPedido`)

Paso 1 – Marcar las decisiones en el código 5.1: D1 `pedido == null`, D2 `isEmpty()` (el `||` se separa en dos condiciones simples), D3 primer `for`, D4 `stock < cantidad`, D5 `!aprobado`, D6 segundo `for`.

Paso 2 – Agrupar en nodos: cada decisión es un nodo; las sentencias seguidas sin decisiones forman un solo nodo; se agrega un nodo de salida único.

Paso 3 – Dibujar el grafo y listar las aristas:



Arista	De → a	Significado
a1	1 → 3	D1 V: pedido nulo
a2	1 → 2	D1 F
a3	2 → 3	D2 V: sin items
a4	2 → 4	D2 F
a5	4 → 5	D3: hay item por validar
a6	4 → 8	D3: no quedan items
a7	5 → 6	D4 V: sin stock
a8	5 → 7	D4 F: hay stock
a9	7 → 4	siguiente iteración
a10	8 → 9	D5 V: rechazado
a11	8 → 10	D5 F: aprobado
a12	10 → 11	D6: hay item por descontar
a13	11 → 10	siguiente iteración
a14	10 → 12	D6: no quedan items
a15	3 → 13	`throw`
a16	6 → 13	`throw`
a17	9 → 13	`return`
a18	12 → 13	`return`

Figura 6.3: Grafo de flujo de `procesarPedido`

Paso 4 – Calcular $V(G)$ de tres formas, para comprobar que el grafo está bien:

$$V(G) = 18 - 13 + 2 = 7 \qquad V(G) = 6 + 1 = 7 \qquad V(G) = 7 \text{ regiones}$$

Se necesitan **7 casos**.

Paso 5 – Derivar los caminos. Se toma un camino base (el flujo normal) y se invierte una decisión a la vez:

Tabla 6.6: Caminos independientes

Camino	Cómo se obtuvo	Nodos
C6	base: 1 item, pago aprobado	1-2-4-5-7-4-8-10-11-10-12-13
C1	invertir D1 (pedido nulo)	1-3-13
C2	invertir D2 (sin items)	1-2-3-13
C3	invertir D4 (sin stock)	1-2-4-5-6-13
C5	invertir D5 (rechazado)	1-2-4-5-7-4-8-9-13
C4	2 vueltas; sin stock en la 2.ª	1-2-4-5-7-4-5-6-13
C7	3 vueltas en cada bucle	1-2-4-(5-7-4)×3-8-10-(11-10)×3-12-13

Con C6, C1, C2, C3 y C5 ya se recorren las 18 aristas. C4 y C7 completan los 7 caminos variando las iteraciones de los bucles, porque el camino con cero iteraciones es imposible: si D2 dejó pasar, hay items.

Paso 6 – Forzar cada camino. D1, D2, D3 y D6 dependen de los *datos*; D4 y D5 dependen de lo que respondan los sistemas externos, así que **se fuerzan con stubs de Mockito**. Luego, `verify` demuestra por qué nodos pasó la ejecución y por cuáles no. La calculadora se simula con un valor fijo para aislar `procesarPedido`.

Tabla 6.7: Casos de caja blanca (P1 = 10 Bs, P2 = 2 Bs, P3 = 80 Bs)

Caso	Datos	Stubs que fuerzan el camino	Esperado y verificación
C1	`null`	–	error; `verifyNoInteractions` (ningún mock usado)
C2	sin items	–	error; ningún mock usado
C3	P1 × 5	stock → 4	error de stock; la pasarela nunca se usa
C4	P1 × 1, P2 × 3	stock P1 → 10, P2 → 0	error "…P2"; se consultaron los dos stocks
C5	P1 × 2	stock → 2; cobrar → rechazado	RECHAZADO; `never()` descuenta ni guarda
C6	P3 × 1, VIP	stock → 1; descuento → 4; cobrar(76) → aprobado	PAGADO, total 76; descuenta 1; mensaje exacto
C7	P1, P2 × 5, P3 × 2	stock → 100; descuento → 9; cobrar(171) → aprobado	`times(3)` en consultas y descuentos

Ficha C5 – Pago rechazado

Origen / técnica	Caja blanca, camino C5 = 1-2-4-5-7-4-8-9-13 (D4 = F, D5 = V).
Datos de entrada	Pedido con P1 × 2 (subtotal 20 Bs); REGULAR.
Stubs (mocks)	`stockDisponible("P1")` → 2; `calcularDescuento(20, REG)` → 0; `cobrar(EMAIL, 20)` → `rechazado("Sin fondos")`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado RECHAZADO.
Verificaciones	Pasó por el nodo 9: `verify(notificador).enviar(EMAIL, "Pago rechazado: Sin fondos")`. No pasó por 11 ni 12: `never().descontarStock`, `never().guardar`.
Resultado obtenido	Coincide con lo esperado: **APROBADO**

```
Pedido pedido = new Pedido("X", EMAIL, List.of(new ItemPedido(P1, 2)));   // D1=F, D2=F
when(inventario.stockDisponible("P1")).thenReturn(2);                      // nodo 5: D4=F
when(calculadora.calcularDescuento(20.0, TipoCliente.REGULAR)).thenReturn(0.0);
when(pasarela.cobrar(EMAIL, 20.0)).thenReturn(ResultadoPago.rechazado("Sin fondos")); // nodo 8:
    D5=V

Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.REGULAR);

assertEquals(EstadoPedido.RECHAZADO, resultado.getEstado());               // nodo 9
verify(notificador).enviar(EMAIL, "Pago rechazado: Sin fondos");           // nodo 9
verify(inventario, never()).descontarStock(anyString(), anyInt());         // NO nodo 11
verify(pedidoRepository, never()).guardar(any());                          // NO nodo 12
```

Código 6.3: Camino C5: cada línea corresponde a un nodo del grafo

Paso 7 – Medir. La cobertura se mide con JaCoCo (`./run.sh cobertura`); los resultados están en el capítulo 7. Con el mismo procedimiento se diseñaron 5 casos de caja blanca (K1 a K5, $V(G) = 5$) para `calcularDescuento`.

Capítulo 7

Resultados

Tabla 7.1: Ejecución de las pruebas (`./run.sh`, 0,8 s)

Categoría	Pruebas en verde
Ejemplos de la API (E01–E10)	30
Caja negra (CD-01 a 15, R1–R4, L1)	20
Caja blanca (C1–C7, K1–K5)	12
Construcción paso a paso (V1, V4, V5)	3
Total	**65**

+ 3 versiones en rojo a propósito (V2, V3, V6), que se ejecutan con `./run.sh rojos`.

Tabla 7.2: Ramas cubiertas de `PedidoService` (JaCoCo)

Línea	Código	Solo caja negra	Solo caja blanca	Todas
42	`if (pedido == null || isEmpty())`	3/4	4/4	4/4
47, 49, 59, 65	`for`, `if` stock, `if` pago, `for`	8/8	8/8	8/8
	Total	**11/12**	**12/12**	**12/12**

Hallazgos

- La caja negra no cubrió la rama `pedido == null` porque la SRS no la menciona (OBS-01). La encontró la caja blanca (C1).
- Al revés, al introducir un error a propósito (< cambiado por <=), lo detectó el caso de valor límite **L1** de la caja negra.
- Por eso **las dos técnicas se complementan**. Juntas alcanzan el 100 % de ramas y cumplen el RNF-02.

Capítulo 8

Conclusiones y recomendaciones

1. Mockito permite probar una clase **aislada** de la base de datos, el banco y el correo: 65 pruebas en menos de un segundo sin tocar sistemas reales.
2. Su arquitectura explica sus reglas. Toda llamada a un mock va a su `MockHandler`, y `when()` usa la última llamada registrada; de ahí salen, por ejemplo, la regla "todos los argumentos con matchers o ninguno" y el uso de `doXxx()` en métodos `void`.
3. La SRS y el SDD hicieron posible el trabajo: la SRS dio los requisitos verificables para la caja negra, y el SDD, con interfaces inyectadas, los puntos donde colocar los mocks.
4. Diseñar los casos **paso a paso** (clases, límites, reglas y caminos, con el resultado esperado calculado a mano) hace que cada prueba se pueda justificar y auditar. Además, reveló 4 vacíos en los requisitos.
5. Para la **auditoría de sistemas**, las pruebas automatizadas, la matriz de trazabilidad y el reporte de cobertura son evidencia objetiva y repetible.

Recomendaciones: ejecutar las pruebas en integración continua, complementarlas con pruebas de integración y corregir los vacíos OBS-01 a OBS-04 en la SRS.

Referencias

[1] Mockito. *Documentación oficial*. https://site.mockito.org

[2] JUnit Team. *JUnit User Guide*. https://docs.junit.org

[3] Meszaros, G. (2007). *xUnit Test Patterns*. Addison-Wesley.

[4] McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4).

[5] Pressman, R. y Maxim, B. (2020). *Ingeniería del software: un enfoque práctico*, 9.ª ed. McGraw-Hill.

[6] Myers, G. et al. (2011). *The Art of Software Testing*, 3.ª ed. Wiley.

[7] IEEE 830-1998 / ISO/IEC/IEEE 29148:2018. *Requirements specification*.

[8] IEEE 1016-2009. *Software Design Descriptions*.

[9] ISO/IEC/IEEE 29119. *Software Testing*.

Capítulo A

Proyecto y ejecución

```
ejemplos-mockito/
|-- src/main/java/.../   modelo, puertos (interfaces), servicio (PedidoService,
    CalculadoraDescuentos)
`-- src/test/java/.../
    |-- basico/       E01 ... E10   ejemplos de la API y de la arquitectura
    |-- pasoapaso/    Paso1 ... Paso6   la prueba de RF-05 de rojo a verde
    |-- cajanegra/    clases de equivalencia, valores límite, tabla de decisión
    `-- cajablanca/   camino básico de procesarPedido y calcularDescuento

./run.sh               # ejecuta las pruebas (solo necesita JDK 21+)
./run.sh rojos         # muestra las versiones en rojo del paso a paso
./run.sh cobertura     # genera el reporte JaCoCo en target/cobertura/index.html
```

Código A.1: Estructura y comandos